COS30031 • GAMES PROGRAMMING • ASSIGNMENT 2

ZERO-DAY: LAST LINE

Development & Testing Report

A top-down defence game built to dramatize Qualys's Detect → Validate → Remediate exposure-management loop.

3 LEVELS	**7/7** TESTS PASSING	**14** SPRITES DRAWN	**8** PHYSICS MATERIALS

Diesel, Shashimal, Sean, & Viresh — Unreal Engine 5.8, C++

29 September 2026

Itch.io: Zero-Day: Last Line by ZeroDayz

GitHub: Commits · 105910175/GameMakersAssignment2

YouTube: https://youtu.be/g58__ZikbKQ



Contribution Summary

Category	File(s)	Contributor(s)
Core gameplay	ZeroDayGameMode.h / .cpp — level flow, resource inventory, win/lose (Viresh added audio hooks)	Diesel, Viresh
Core gameplay	WallActor.h / .cpp — wall health, status, breach countdown (Viresh added an audio hook)	Diesel, Viresh

Core gameplay	BreachableComponent.h / .cpp — reusable health/status tracker	Diesel Williams
Core gameplay	ThreatSpawnerActor.h / .cpp — spawns threats, applies level difficulty (Viresh added an audio hook)	Diesel Williams
Core gameplay	ThreatPawn.h / .cpp — the attacker: movement, severity, decoy status (Viresh added an audio hook)	Diesel Williams
Core gameplay	CityTaskActor.h / .cpp — Power/Food/Medical stations (Viresh added audio hooks)	Diesel Williams
Core gameplay	CityTaskComponent.h / .cpp — the decaying condition meter (Viresh added audio hooks)	Diesel, Viresh
Core gameplay	CooldownComponent.h / .cpp — shared timer (walls, stations, spawners, surges)	Diesel Williams
Core gameplay	DetectionComponent.h / .cpp — Insta Scan and Validate (Viresh added an audio hook)	Diesel, Viresh
Core gameplay	ResourcePickup.h / .cpp — physics-simulated pickups (Viresh added an audio hook)	Diesel, Viresh
Core gameplay	ZeroDayProjectile.h / .cpp — the player's shot (Viresh added an audio hook)	Diesel Williams
Core gameplay	ZeroDayHUD.h / .cpp — every on-screen panel, bar, alert and marker	Diesel Williams
Core gameplay	ZeroDayPhysics.h / .cpp — the 8 physics material specs	Diesel Williams
Core gameplay	ZeroDaySprites.h / .cpp — the lay-flat sprite helper	Diesel Williams
Core gameplay	ZeroDayLevelConfig.h / .cpp — the per-level difficulty table (levels 1–3)	Diesel Williams
Core gameplay	ZeroDayTypes.h — shared enums (wall status, severity, resource type...)	Diesel Williams
Core gameplay	ZeroDayAssetGen.h / .cpp — the sprite/physics-material import commandlet	Diesel Williams
Core gameplay	ZeroDayDevDemo.h / .cpp — scripted screenshot demo tool (dev-only)	Diesel Williams
Core gameplay	Tests/ZeroDayEconomyTests.cpp, ZeroDayLevelTests.cpp, ZeroDaySizeTests.cpp, ZeroDayTestWorld.h — the 12 automation tests	Diesel Williams

Player character	ZeroDayCharacter.h / .cpp — movement, camera, aiming, firing, Interact	Diesel, Viresh
Audio system	Audio/ZeroDayAudioComponent.h / .cpp — the audio component itself	Viresh
Sprite pipeline	Art/generate_sprites.py — draws every sprite	Diesel Williams
Sprite pipeline	Content/Sprites/ — 28 files (T_* textures + S_* Paper2D sprites)	Diesel Williams
Physics materials	Content/Physics/PM_Wall, PM_Threat, PM_Projectile, PM_Intel, PM_Patch, PM_Food, PM_Power, PM_Medical	Diesel Williams
SFX	Content/Audio/SFX/ — 16 sounds: Fire, Scan, Validate, Pickup, SupplyDrop, ProjectileImpact, ThreatDestroyed, WallHit, WallBreached, WallRepaired, StationWarning, StationAttended, StationFailed, Surge, LevelWon, LevelLost	Viresh
Music	Content/Audio/Music/BackgroundMusic	Viresh
Player animation	Content/PlayerAnimations/ — walking flipbooks (FB_Player_Walk_Up / Down / Left / Right) and the pixel_character_dark_blue sprite sheet	Viresh
Player animation	Content/Python/create_walk_flipbooks.py (and create_player_flipbook(s).py) — scripts that build the flipbooks	Viresh
Environment art	Content/Environments/ — T_DesertGround, T_DesertGround_Sprite, T_DesertMap, M_DesertGround, M_DesertMaps	Viresh
Level design	Content/Levels/L_Level1_Detect.umap — the playable map	Viresh
Blueprints	Content/Blueprints/BP_ZeroDayCharacter, BP_ZeroDayGameMode	Viresh
Project config	Config/DefaultEngine.ini, DefaultGame.ini, DefaultInput.ini	Viresh, Diesel
Project config	Config/DefaultEditor.ini	Viresh
Project config	ZeroDayLastLine.uproject, ZeroDayLastLine.Build.cs	Viresh, Diesel
Documentation	README.md, this report, the code walkthrough	Diesel Williams
Demo video	Video editing — cutting and assembling the demo footage	Shashimal
Demo video	Demoing the game — presenting and playing through the levels for the video	Viresh

SECTION 01

Executive Summary

Zero Day: Last Line is a top down 2D survival game built in Unreal Engine 5.8. It takes Qualys's exposure management workflow and turns it into a playable experience where you scan the perimeter, verify what is actually there, and fix the issues before time runs out. The brief was to create a game that connects ETM, Agent Insta, Agent Val / TruConfirm, and TruRisk Eliminate in a way that stays true to the Qualys products. The project was also expected to be fully developed, tested, and validated as a working build rather than just a concept or paper design.

> **WHAT THIS REPORT COVERS**
>
> Three difficulty levels, Detect, Validate, and Remediate, each with increasing threat pressure.
>
> A resource economy built around Intel, Patches, Food, Power, and medical supplies that shapes every player decision. A fully custom sprite pipeline built from scratch using Python and an Unreal command let, along with eight hand tuned physics materials.
>
> Seven automation tests covering the level flow and resource economy, running heedlessly every time something changes. One unresolved rendering issue that has been documented honestly in Section 7.

SECTION 02

The Brief, and How It Maps to the Game

Qualys presents modern exposure management as an ongoing process. It involves identifying exposures across the environment, checking which findings can be exploited, prioritizing them based on business risk, and then addressing them. Increasingly, this process also involves autonomous, and agent assisted tools (Qualys, Inc., 2025; Qualys, Inc., 2026a; Qualys, Inc., 2026b). The game reflects this process directly rather than just describing it in theory. Any references to Qualys products are kept conceptual, meaning they represent a particular capability rather than making claims about how the actual products work.

Qualys concept	In game mechanic	Design intent
ETM (unified exposure view)	Status panel showing the health of every wall and station in one place	Nothing about the threat is hidden. The only information missing is from threats that have not been scanned yet.

Agent Insta (fast, broad detection)	Insta Scan (Q) is free and instant, and flags every incoming threat	Detection should be easy and accessible, but it does not tell the player whether a threat is actually real.
Agent Val / TruConfirm (validation)	Validate (F) costs Intel and reveals the threat severity and whether it is a false positive	Confirming whether something is actually exploitable requires a resource, representing the extra effort involved in triage.
TruRisk Eliminate (remediation)	Fix a wall (E) costs a Patch Kit and repairs the breach	Remediation is the outcome of correctly identifying and prioritising a real threat.
Risk prioritisation	Threat severity (Low, Medium, Critical) affects damage, health, and colour coding	Not every exposure presents the same level of risk, so players need to prioritise which threats to deal with first.

The false-positive mechanic (a threat that turns out to be harmless once validated) was added deliberately: it's what makes Insta Scan feel incomplete on its own, and gives Validate a real job to do rather than being a formality.

SECTION 03

System Architecture

The game is structured around a small number of focused Actor classes that are managed by a single AZeroDayGameMode. Reusable ActorComponents handle behaviours that are shared between different classes, such as the decaying meter, countdown timer, and scan and validation tools. This makes each class easier to understand and maintain on its own. For example, a wall does not need to know how a station's grace period works. Both can simply use the same UCooldownComponent to handle that behaviour.

Class	Role
AZeroDayGameMode	Owns level rules, the resource inventory, survivors, win/lose, and builds the city if the map is empty
AWallActor	One of four perimeter walls: health, Safe/Threatened/Breached status, the breach countdown
AThreatPawn	An attacker walking toward its target wall; severity and decoy status set at spawn
ACityTaskActor	A Power / Food / Medical station with a decaying condition meter

AResourcePickup	A physics-simulated resource that bounces and slides to a stop
UDetectionComponent	The player's Insta Scan and Validate abilities
AZeroDayCharacter	The player: movement, aiming, firing, and the Interact (E) key
AZeroDayHUD	Every on-screen element, drawn directly to the canvas each frame

SECTION 04

Design Decisions and Iteration

The build went through a deliberate second pass after the core loop was already working end-to-end, driven by direct playtesting feedback: the city felt cramped, the walls read as thin lines rather than a barrier, the player sprite was oversized next to them, and the HUD was functional but visually flat. Each of these was treated as a real design problem, not a cosmetic tweak.

✦ 4.1 A city built to be walked, not stood in

The playable area was made roughly twice as large in each direction, and the four walls were connected at the corners so they now feel like one continuous barrier rather than four separate strips. This makes defending the wall a much more meaningful spatial decision. Going to a station now means leaving part of the wall exposed for several seconds, something the smaller original layout did not really force the player to consider.

✦ 4.2 A player sized for the space

The player capsule and mesh were scaled down to 65% so the character felt more human sized compared to the larger walls. This also revealed a real logic bug in the wall repair system. The distance check was measuring from the player to the centre of the wall, which meant that on a wall stretching across most of a city side, a player standing near either end could incorrectly be told they were too far away to repair it. The fix was made in AWallActor::GetDistanceToPoint, which now measures the distance to the nearest point along the wall instead. It was the kind of bug that only became obvious once the world was scaled up to feel more realistic.

✦ 4.3 A HUD that looks designed, not default

Each panel now has its own themed frame, with an accent coloured border and small corner brackets that match the visual style of the wall status colours. The health and condition bars were also changed to feel more like segmented gauges rather than simple fill bars. The level timer now pulses and gradually grows during the final fifteen seconds, making the countdown feel more noticeable. Screenshots of the final result can be found in Section 5.

SECTION 05

Testing and Validation

Two complementary methods were used throughout development: automated regression tests for rules that should never silently break, and headless, scripted screenshot captures for anything that needed to be judged visually. Both methods run without opening the interactive editor, using UnrealEditor Cmd.exe (Epic Games, 2026c). This made it practical to recheck the entire game after every significant change instead of relying on manual spot checks.

✦ 5.1 Automated tests

Seven tests were created using Unreal's Automation Test Framework (Epic Games, 2026c). Each test runs in a real, although minimal, game world rather than using mocked objects, allowing them to test the actual game mode, walls, and components. All seven tests currently pass.

Test	What it checks	Result
ZeroDay.Levels.Config	Checks that the per level difficulty table loads correctly	PASS
ZeroDay.Levels.FullPlaythrough	Checks that Levels 1 to 3 can be completed in sequence	PASS
ZeroDay.Levels.LoseAndRestart	Checks that losing and then restarting resets the game state correctly	PASS
ZeroDay.Economy.Resources	Checks that resources can be added and spent, while preventing overspending	PASS
ZeroDay.Economy.FixWall	Checks that AttemptFix() only succeeds when a wall has been breached and a Patch kit is available	PASS

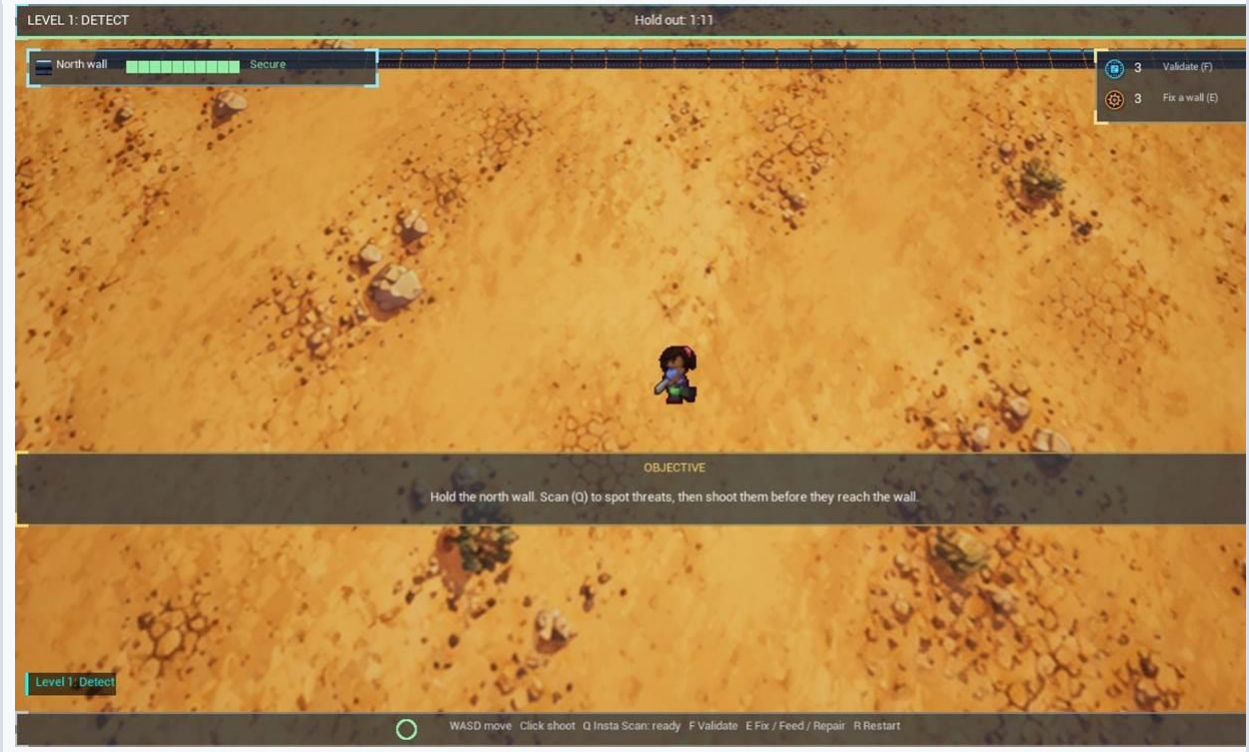
ZeroDay.Economy.Survivors	Checks that an unattended station costs a survivor and that the level ends when there are no survivors remaining	PASS
ZeroDay.Economy.ScanAndValidate	Checks that scanning is free, while validation costs Intel and reveals severity and false positives	PASS

Command used to run the suite headlessly:

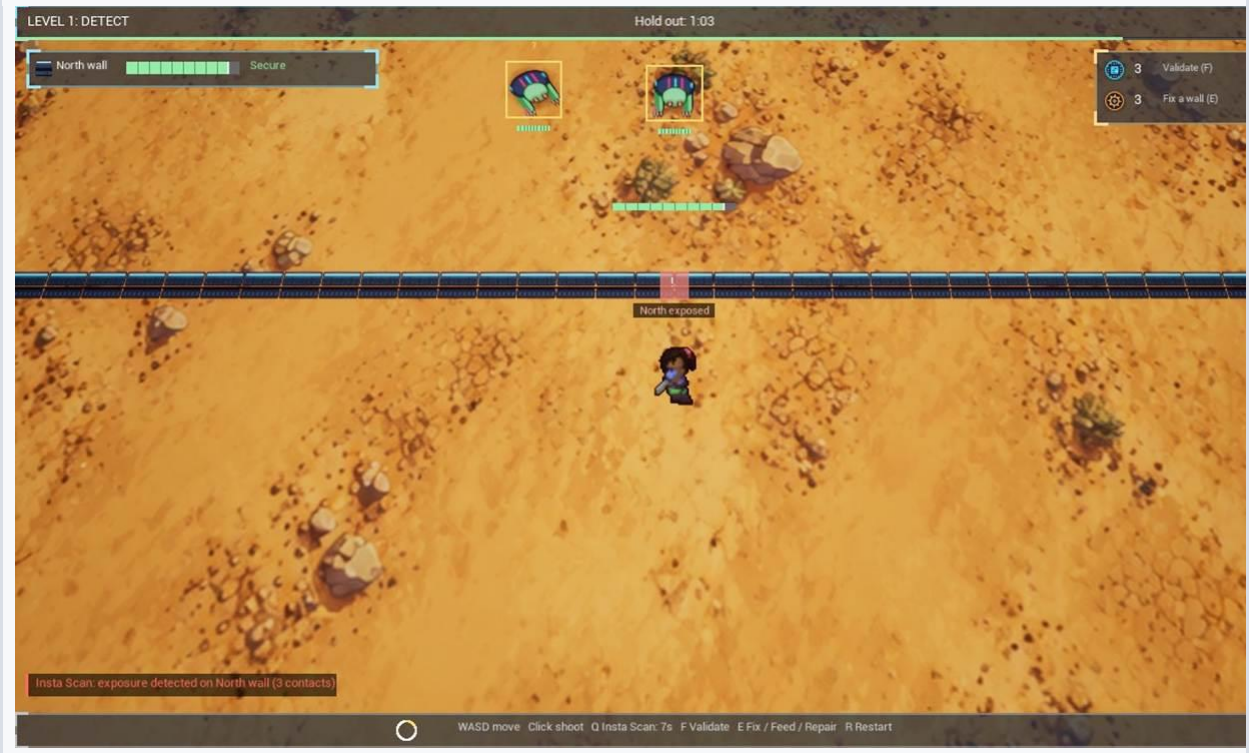
```
UnrealEditor-Cmd.exe ZeroDayLastLine.uproject -ExecCmds="Automation RunTests ZeroDay;
Quit" -nullrhi -unattended
```

✦ 5.2 Visual verification — the level flow

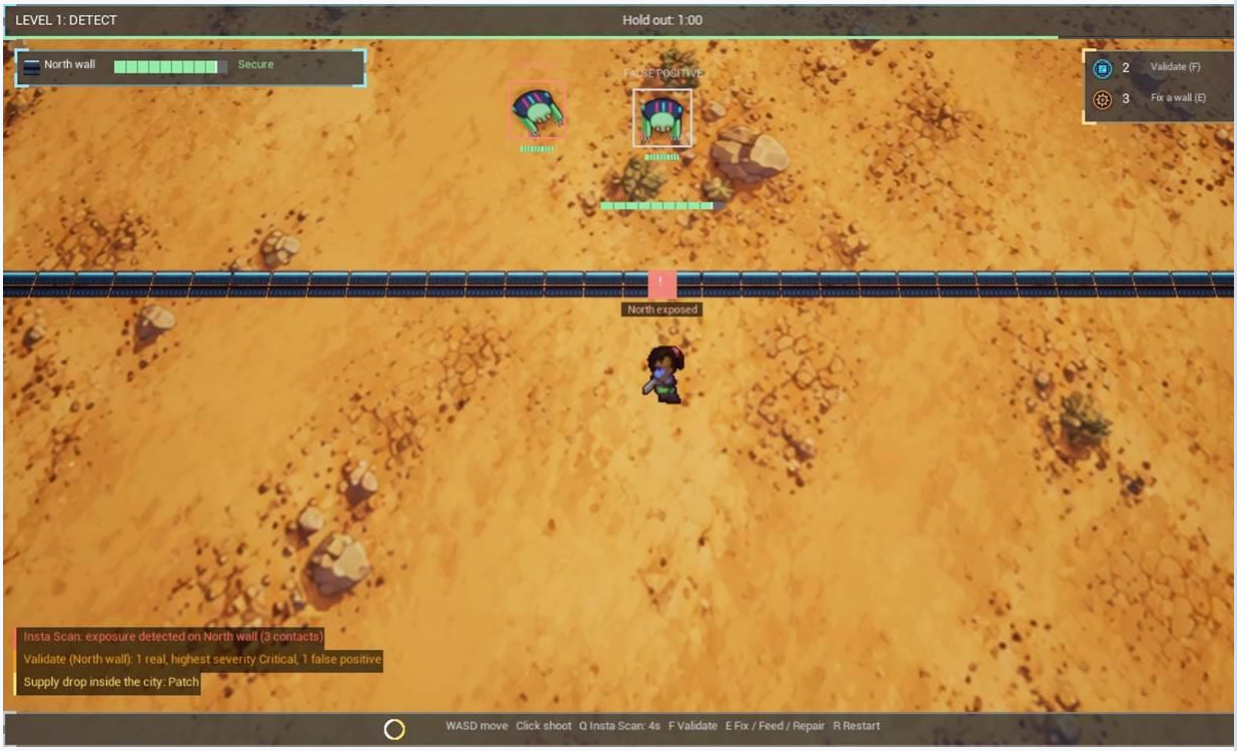
A dev only WorldSubsystem, active only when launched with the ZeroDayDemo flag and compiled out of shipping builds, runs through a full playthrough automatically. It spawns threats, scans and validates them, breaches a wall, completes the level, and captures a screenshot at each milestone using FScreenshotRequest. The sequence below comes from that run, rendered offscreen at 1280 × 720.



Level 1: Detect — objective banner and the redesigned status/resource panels



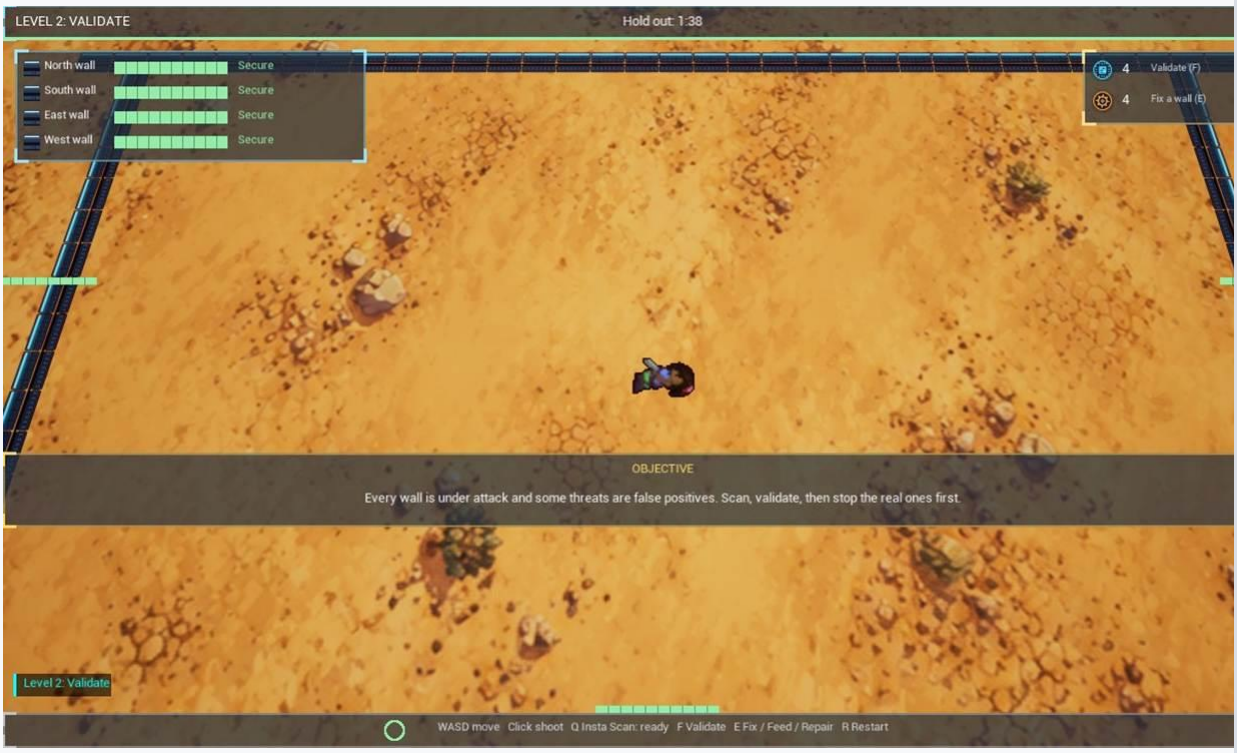
After Insta Scan (Q) — inbound threats flagged but not yet identified



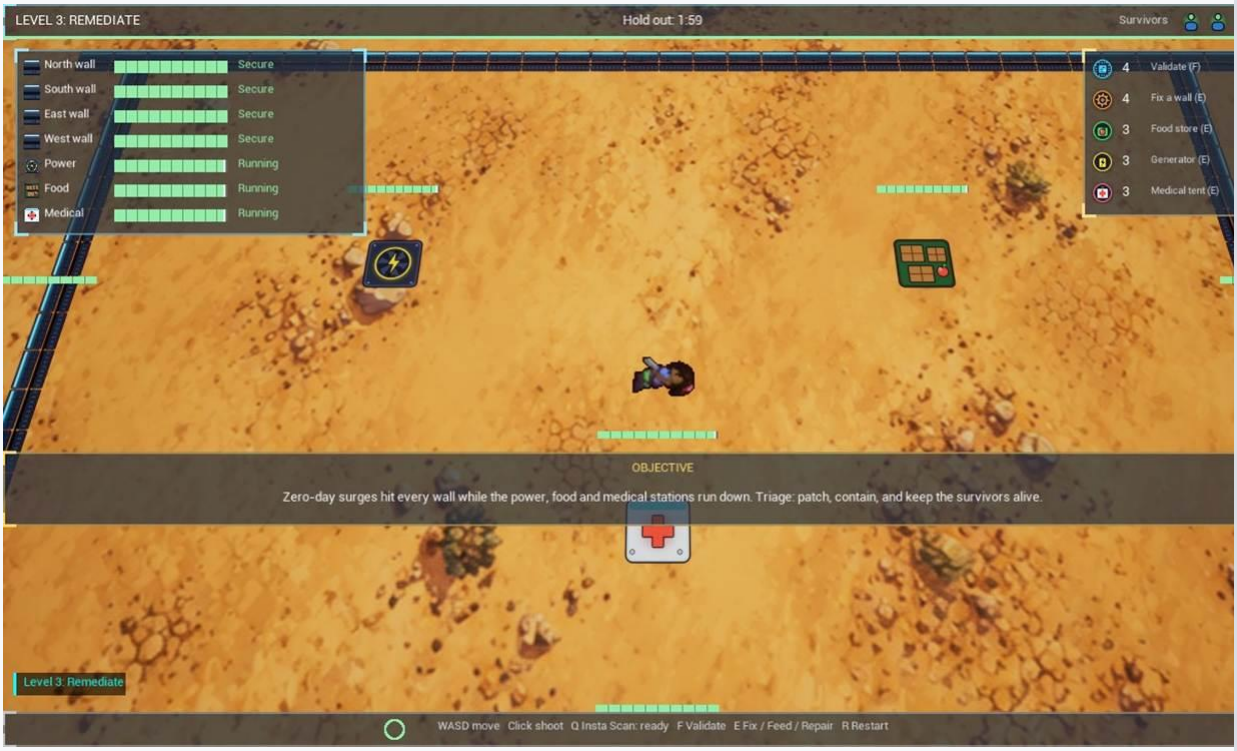
After Validate (F) — severity and false-positive status revealed



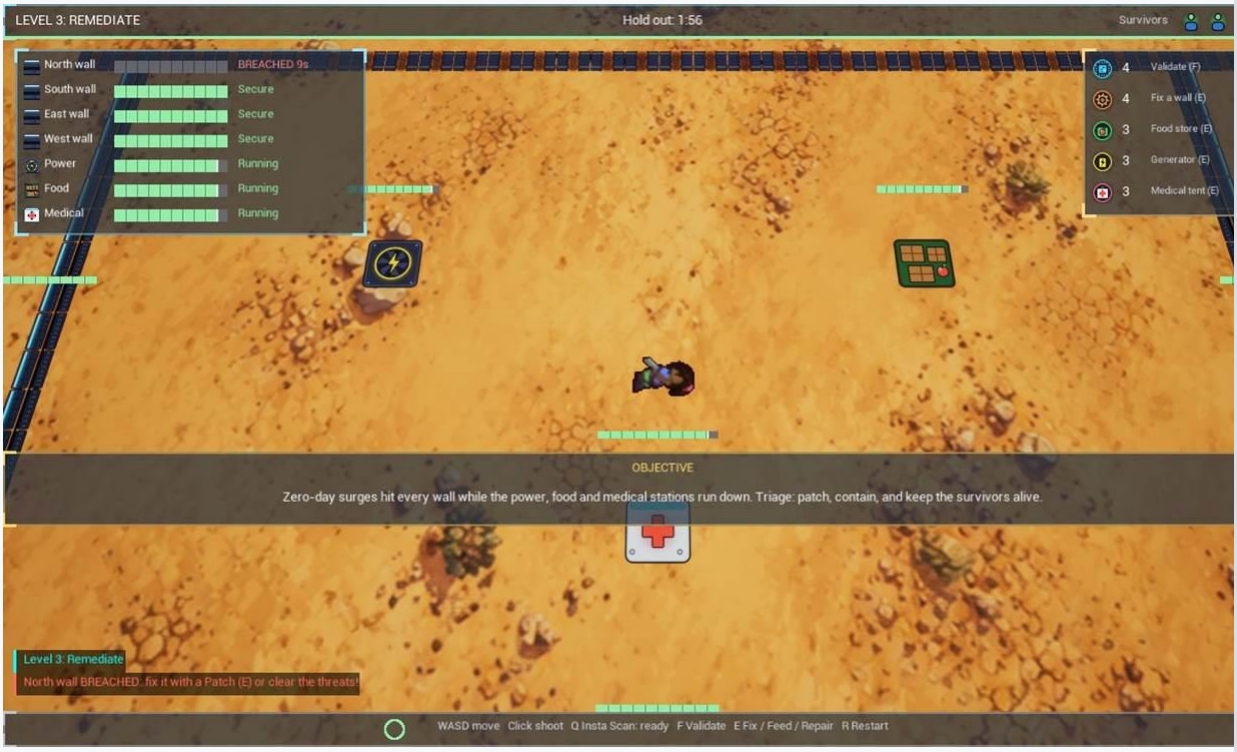
Level 1 complete — the framed, bracketed results panel



Level 2: Validate — all four walls active, threats from multiple sides



Level 3: Remediate — survivors, stations and all four walls tracked at once



A breach on the north wall — pulsing countdown and a styled alert

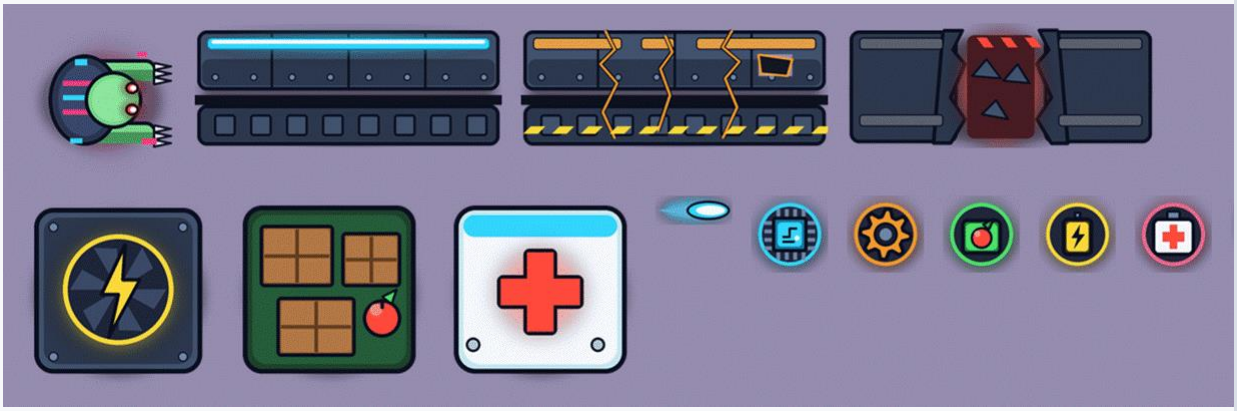


All three levels cleared — the end screen's divider and framed stats panel

SECTION 06

Art Pipeline, in Brief

All of the current sprite art is generated rather than drawn by hand. A Python script at Art/generate_sprites.py, built with Pillow, creates every sprite, while a small Unreal editor commandlet imports the resulting PNGs as real Texture2D and Paper2D sprite assets (Epic Games, 2026a). It also creates eight tuned UPhysicalMaterial assets (Epic Games, 2026b), which give each resource type its own weight and bounce. The full pipeline, including the code, is covered in the companion Code Walkthrough document, so this report focuses on the final result.

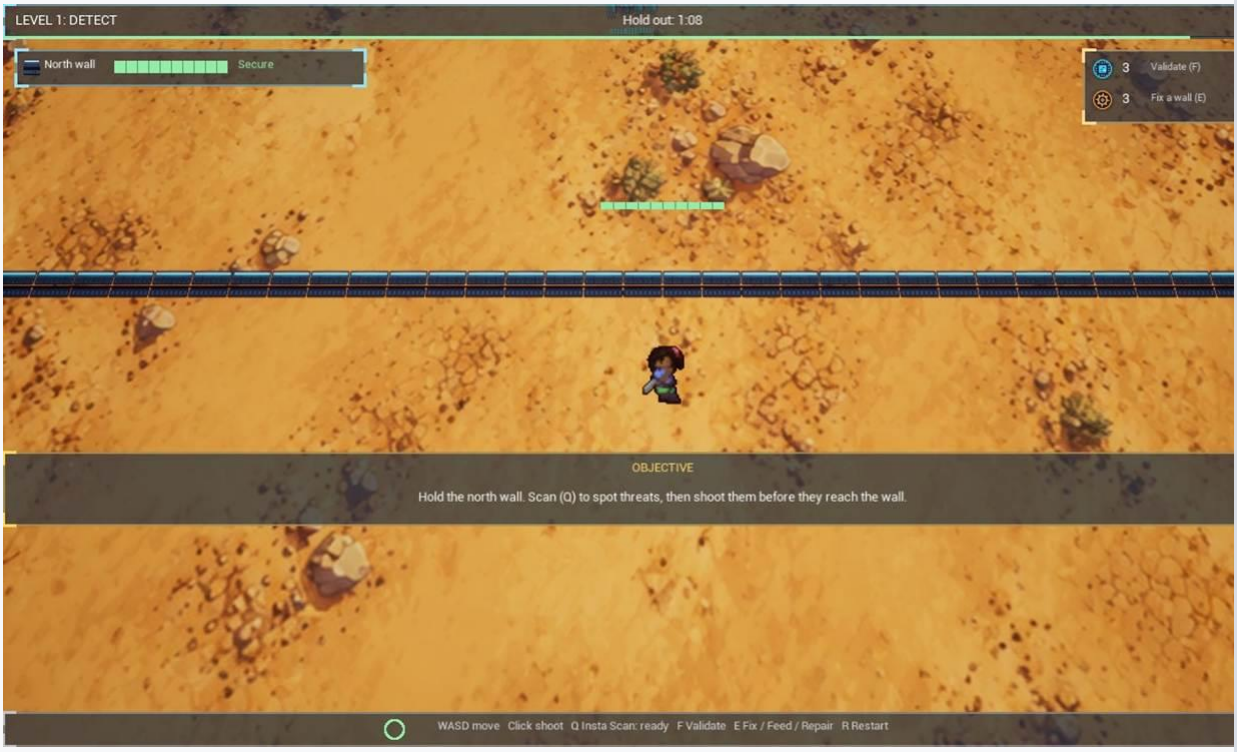


The full generated sprite set — walls (three damage states), stations, threat, pickups and HUD icons

SECTION 07

Known Issues

The wall and station sprites that failed to appear in earlier headless screenshots now render correctly in the latest run. The close-up below shows the wall drawn as a continuous row of tiles with the player standing beside it, and the level 3 screenshots earlier in this report show all four walls and the three stations on screen.



The player standing beside the north wall, drawn as a continuous row of sprite tiles

> **STILL OPEN**
>
> Wall and station collision bodies are still larger than their drawn art (for example a 192-unit-deep wall body against roughly 81 units of artwork), because they were authored assuming one pixel per unit.
>
> The project's sprites actually draw at 2.56 pixels per unit. The threat's hit sphere has been corrected to match, but the wall and station bodies have not, to avoid changing how the game plays.

The likely next step is to resize those collision bodies to match the artwork, which would make walls and stations feel less bulky to walk around, and to re-check the level 3 balance afterwards.

SECTION 08

Reflection and Next Steps

The project's strongest result is that the Detect → Validate → Remediate loop is not just present, but genuinely important to how the game works. Skipping Validate is possible, but it comes with more risk, which is exactly the trade off the brief asks the game to communicate. The automated test suite also proved its value during this development pass. When the city was enlarged, it broke an existing test assumption about Validate's range, and the test caught the issue straight away instead of it becoming a confusing bug later.

✦ Diagnose and fix the wall and station rendering issue described in Section 7

✦ Add automated coverage for pickup physics, including bounce ordering by material, as well as supply and threat drops

✦ Add detailed, line by line comments to the earliest written files so they match the rest of the codebase

✦ Replace the placeholder generated art with the level artist's final sprites using the same import pipeline

References

References

Epic Games (2026a) Automation Test Framework in Unreal Engine. Unreal Engine 5.8 Documentation. Available at: dev.epicgames.com/documentation/en-us/unreal-engine/automation-test-framework-in-unreal-engine (Accessed: 26 September 2026).

Epic Games (2026b) Paper 2D Overview in Unreal Engine. Unreal Engine 5.8 Documentation. Available at: dev.epicgames.com/documentation/en-us/unreal-engine/paper-2d-overview-in-unreal-engine (Accessed: 28 September 2026).

Epic Games (2026c) Physical Materials User Guide for Unreal Engine. Unreal Engine 5.8 Documentation. Available at: dev.epicgames.com/documentation/unreal-engine/physical-materials-user-guide-for-unreal-engine (Accessed: 30 September 2026).

Qualys, Inc. (2025) Introducing TruConfirm for Enterprise TruRisk Management: Automated Exposure Validation. Qualys Blog, 15 October. Available at: blog.qualys.com/.../introducing-truconfirm-for-enterprise-trurisk-management-automated-exposure-validation (Accessed: 27 September 2026).

Qualys, Inc. (2026a) Meet Agent Val: Closing the Validation Gap in Exposure Management at Machine Speed with Agentic AI. Qualys Blog, 23 March. Available at: blog.qualys.com/.../meet-agent-val-closing-the-validation-gap... (Accessed: 2 October 2026).

Qualys, Inc. (2026b) Zero-Day Remediation Meets Operational Resiliency. Qualys Blog, 3 August. Available at: blog.qualys.com/.../zero-day-remediation-operational-resiliency-trurisk-eliminate (Accessed: 5 October 2026).

Swinburne University of Technology (2026) COS30031 Games Programming: Assignment 2 Brief. [Unpublished course material]. Melbourne: Swinburne University of Technology (Accessed: 29 September 2026).